

Manipulera tecken

Många program behöver manipulera tecken i strängar direkt för att utföra någon viss operation. Man kan accessera tecknen enkelt via operatören `[]` eller metoden `at()` och på så vis direkt använda de `char:s` som stängen är uååbyggd av. Det är dock arbetsdrygt och ganska svårt att på bas en `char` veta vad det är för *typ* av tecken. Program kan t.ex. behöva kontrollera om ett tecken är en siffra, en bokstav, whitespace o.s.v. Skall man göra detta manuellt varje gång det behövs blir program mycket svårare att skriva. Det är även skillnad på vilka tecken som kan räknas som alfanumeriska, beroende på vilken *locale* som används. En locale bestämmer olika aspekter på ett systems inställningar, såsom tid, teckentabeller o.s.v. Våra normala skandinaviska tecken å, ä, ö, Å, Ä och Ö är inte bokstäver i alla locales.

Man kan använda sig av en mängd funktioner som finns definierade i headfilen `ctype.h` för att känna igen och klassificera olika tecken. De olika funktionerna är:

- `isalpha()` kontrollerar om ett tecken är en bokstav i den använda locale:n.
- `isupper()` samma som `isalpha()`, men tecknet skall även vara en gemen (liten bokstav).
- `islower()` samma som `isalpha()`, men tecknet skall även vara en versal (stor bokstav).
- `isdigit()` kontrollerar om ett tecken är en siffra (0-9).
- `isxdigit()` kontrollerar om ett hexadecimalt tecken är en siffra (0-9, a-f, A-F).
- `isspace()` kontrollerar om ett tecken är whitespace (space, radbyte `\n`, tabulator `\t` eller vertikal tabulator `\v`).
- `iscntrl()` kontrollerar om ett tecken är ett kontrolltecken.
- `ispunct()` kontrollerar om ett tecken är ett skiljetecken.
- `isalnum()` kontrollerar om ett tecken är alfanumeriskt, d.v.s. antingen `isalpha()` eller `isdigit()`.
- `isprint()` kontrollerar om ett tecken kan skrivas ut (inklusive space).
- `isgraph()` kontrollerar om ett tecken kan skrivas ut (exklusive space).

Alla dessa metoder tar som parameter en `int`, men man kan förstås även skicka en `char`. Alla returnerar en `int` med värdet 0 då det givna teckent inte uppfyllde kriterierna och annars ett annat värde. Vi kan göra ett enkelt program för att kontrollera om en given sträng kan vara ett giltigt socialskyddssignum:

```
#include <iostream>
#include <string>
#include <ctype.h>
#include <stdlib.h>
```

```

bool okSignum (const string & Data) {
    // är längde ok?
    if ( Data.length () != 11 )
        // kan inte vara ett signum
        return false;

    // först skall vi ha 6 siffror
    for ( int Index = 0; Index < 6; Index++ ) {
        if ( ! isdigit ( Data[Index] ) ) {
            // ingen siffra
            return false;
        }
    }

    // nästa skall vara ett '-'
    if ( Data[6] != '-' )
        // inget '-'
        return false;

    // sedan 4 alfanumeriska tecken
    for ( int Index = 7; Index < 11; Index++ ) {
        if ( ! isalnum ( Data[Index] ) ) {
            // ingen siffra
            return false;
        }
    }

    // det kan vara ett signum
    return true;
}

int main (int argc, char * argv[]) {
    // verifiera antal parametrar
    if ( argc != 2 ) {
        cout << "Fel antal parametrar!" << endl;
        cout << "Användning: " << argv[0] << " signum" << endl;
        return EXIT_FAILURE;
    }

    string Signum = argv[1];

    // kolla ifall det är ett signum
    if ( okSignum ( Signum ) ) {
        cout << Signum << " kan vara ett giltigt socialskyddssignum." << endl;
    }
    else {
        cout << Signum << " är inte ett giltigt socialskyddssignum." << endl;
    }
}

```

Programmet kontrollerar inte att födelsedatumen är giltig, ej heller att slutdelen är giltig. För det finns det speciella algoritmer som faller utanför detta kompendiums omfång.

Konvertera tecken

Två speciella funktioner kan användas för att konvertera tecken till versaler respektive gemener. Dessa funktioner finns definierade i headefilen `ctype.h` och heter `toupper()` och `tolower()`. Båda tar som argument en `int` (ett tecken) och returnerar det konverterade tecknet. Ifall tecknet inte kunde konverteras returneras tecknet oförändrat. Vi kan göra två funktioner för att konvertera alla tecken i en sträng till versaler respektive gemener:

```
#include <iostream>
#include <string>
#include <ctype.h>
#include <stdlib.h>

string toUpper (const string & Data) {
    string Resultat;

    // reservera korrekt antal tecken
    Resultat.resize ( Data.length () );

    // iterera över hela strängen
    for ( unsigned int Index = 0; Index < Data.length (); Index++ ) {
        Resultat[Index] = toupper ( Data[Index] );
    }

    return Resultat;
}

string toLower (const string & Data) {
    string Resultat;

    // reservera korrekt antal tecken
    Resultat.resize ( Data.length () );

    // iterera över hela strängen
    for ( unsigned int Index = 0; Index < Data.length (); Index++ ) {
        Resultat[Index] = tolower ( Data[Index] );
    }

    return Resultat;
}
```

Vi kan sedan lägga till ett litet huvudprogram för att testa hur dessa funktioner fungerar:

```
int main (int argc, char * argv[]) {
    // verifiera antal parametrar
    if ( argc != 2 ) {
        cout << "Fel antal parametrar!" << endl;
        cout << "Användning: " << argv[0] << " signum" << endl;
        return EXIT_FAILURE;
    }

    string S = argv[1];

    // konvertera och skriv ut
    cout << "Som versaler: " << toUpper ( S ) << endl;
    cout << "Som gemener:  " << toLower ( S ) << endl;
}
```

Kör man programmet kan man t.ex. få följande resultat:

```
% ./Convert "Ett litet test, 123!"
Som versaler: ETT LITET TEST, 123!
Som gemener:  ett litet test, 123!
%
```